



Vector DB 종류와 선정기준

{ Vector • Graph 데이터베이스 }

Vector DBMS의 정의 및 3대 핵심 특징

고차원 벡터의 저장·색인 (인덱싱)

빠른 검색 및 유사성 검색을 위한 특수 DBMS

- 수십~수천 차원 벡터 임베딩 저장 및 질의에 최적화
- 전용 인덱스(HNSW, IVF, PQ 등) 탑재 및 수백만 벡터 중 k개를 밀리초(ms) 단위로 탐색



메타데이터 저장 및 필터링 (유연성)

임베딩 값과 속성 조건을 결합한 동적 검색 지원

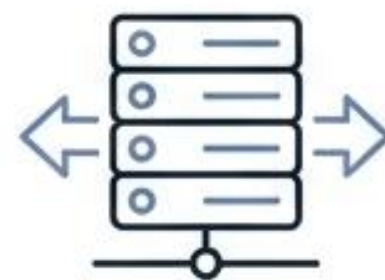
- 벡터 유사도 단독 처리의 한계 극복 (메타 필터 결합 필수)
- 검색 예시: '최근 1년 이내 + 한식 분류 + 가장 유사한 레시피'



시스템 확장성 (Scalability)

데이터 및 사용자 수요 증가에 완벽히 대응

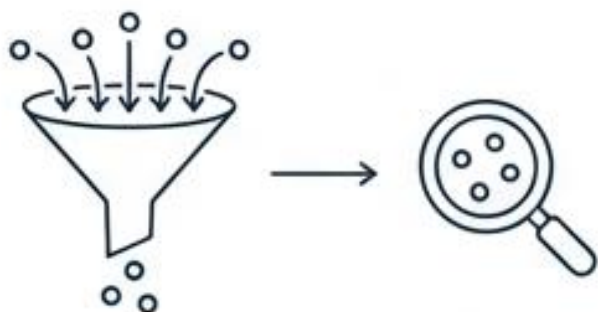
- 수평 확장(Scale-out) 아키텍처 지원
- 대규모 트래픽 환경에서도 일관된 성능 및 응답성 유지



필터링과 ANN의 충돌 구조 (Pre vs Post)

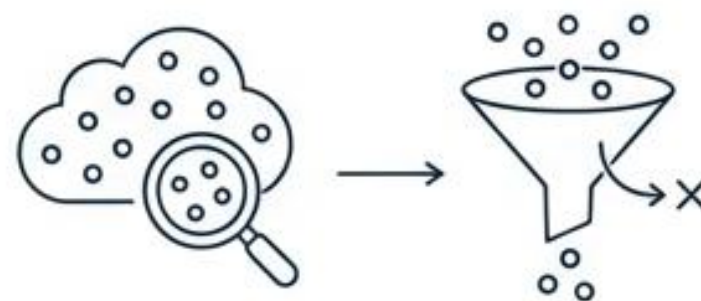
Pre-filtering (사전 필터링)

 작동 방식:	메타데이터 조건 대상 우선 선별 → 이후 벡터 검색 수행
 장점:	조건에 부합하는 정확한 검색 결과 도출 보장
 단점/한계:	조건 부합 대상군이 작을 경우 전수 비교로 전환되어 성능 급락 발생
 적용 환경:	권한, 기간, 문서 유형 등 복잡하고 엄격한 조건이 필수적인 업무



Post-filtering (사후 필터링)

 작동 방식:	벡터 기반 상위 k개 선 검색 → 조건 불일치 건 최종 제거
 장점:	구현이 직관적이며 시스템 적용 및 구성 용이
 단점/한계:	필터링 후 최종 결과가 0건으로 도출될 치명적 위험 존재
 적용 환경:	단순 유사도 중심의 추천 시스템 등 조건 제약이 느슨한 환경



조건이 복잡한 업무(권한·기간·부서)일수록 필터링 성능이 벤치마크의 핵심 지표

Vector DBMS의 등장 배경과 확산 흐름

2020년 이후



다양한 Vector DBMS
아키텍처 개발 및
엔터프라이즈 채택 급증

2022년 11월



ChatGPT 출시 기점,
생성형 AI(RAG) 연계
사용량 폭발적 증가

2023년 이후



오라클 23c 등 상용 RDB의
AI 벡터 유사도 검색 기능
자 흡수 발표

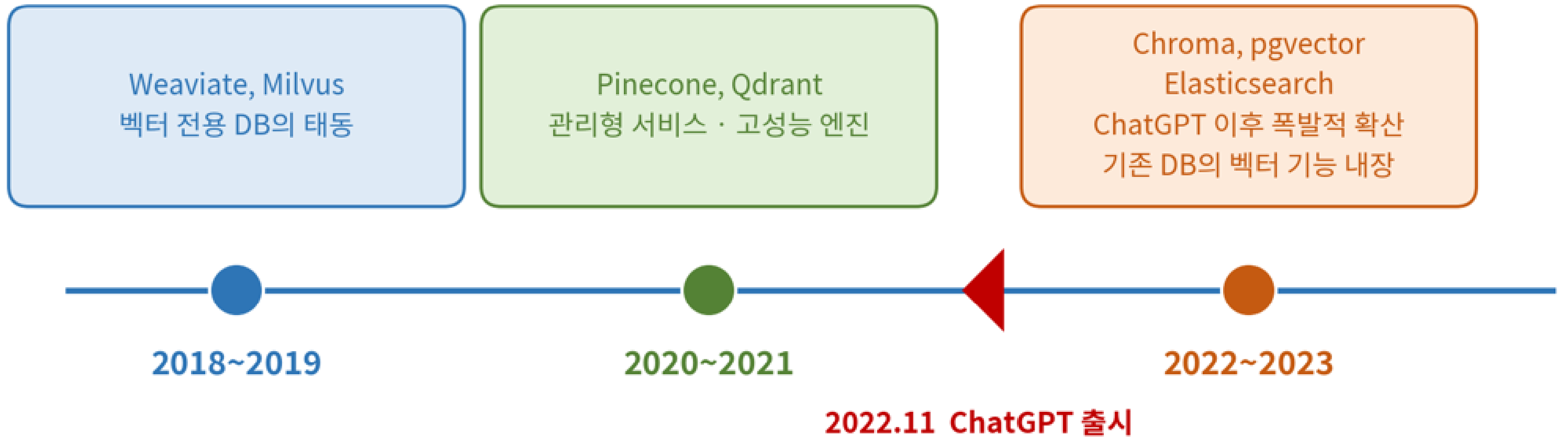
시장 동향

전용 Vector DBMS 시장의 폭발적 성장과 기존 RDB의
벡터 기능 흡수가 동시에 진행되는 과도기적 시장

미래 전망

향후 전용 인프라뿐만 아니라 기존 RDB 생태계 전반으로
벡터 검색 기능이 필수 스펙으로 확대 적용될 전망

Vector DBMS의 등장 배경과 확산 흐름



핵심 요구사항별 권장 DBMS 매핑

비즈니스 요구사항	권장 검토 대상	선정 사유 (핵심 강점)
오픈소스를 선호	Weaviate, Milvus, Chroma	소스 완전 공개, 내부 아키텍처에 맞춘 자유로운 커스터마이징 가능
가격이 낮은 DB 우선	Qdrant	Managed 서비스(SaaS) 도입 및 유지 비용이 타 제품 대비 상대적으로 저렴
대규모 엔터프라이즈급	Pinecone, Milvus	세밀한 역할 기반 접근 제어(RBAC) 기능 제공 및 보안성 우수
단순 문서 기반 검색	Elasticsearch	기존 Inverted Index 기반의 강력한 텍스트 검색 기능에 강점 보유
설치 없이 개인 사용	FAISS	복잡한 DB 설치 불필요, 라이브러리 import만으로 즉시 사용 가능



가용성 및 확장성 측면의 세부 분석 필요 → 이후 제시되는 [4가지 선택 축]을 통해 최종 결정

선택 축 ① - 배포 형태 (On-Premise vs Managed)

평가 항목	On-Premise (직접 구축형)	Managed (완전 관리형)
설치 및 관리	사용자가 하드웨어·소프트웨어 직접 통제	서비스 제공자가 모든 유지보수 전담
보안 통제	조직 내부의 완벽한 보안 설정 및 통제 가능	제공자 환경 의존 (맞춤형 보안 설정 제한)
확장성(Scale)	물리적 하드웨어 신규 구매 및 수동 설정 필요	트래픽에 따른 클라우드 기반 자동 확장 우수
기술 지원	사내 IT 인프라팀 또는 외부 협력업체 리소스 의존	제공자의 전문적이고 신속한 기술 지원 제공
패치 업데이트	사용자가 직접 버전 관리 및 보안 패치 적용	제공자가 무중단 자동 업데이트 수행
벤더 종속성	특정 벤더 종속 없이 시스템 전체 아키텍처 통제	서비스 제공자 플랫폼 생태계에 종속 가능
데이터 소유권	로컬 스토리지 저장으로 데이터 소유권 완전 확보	클라우드 저장에 따른 소유권 및 접근 우려 존재

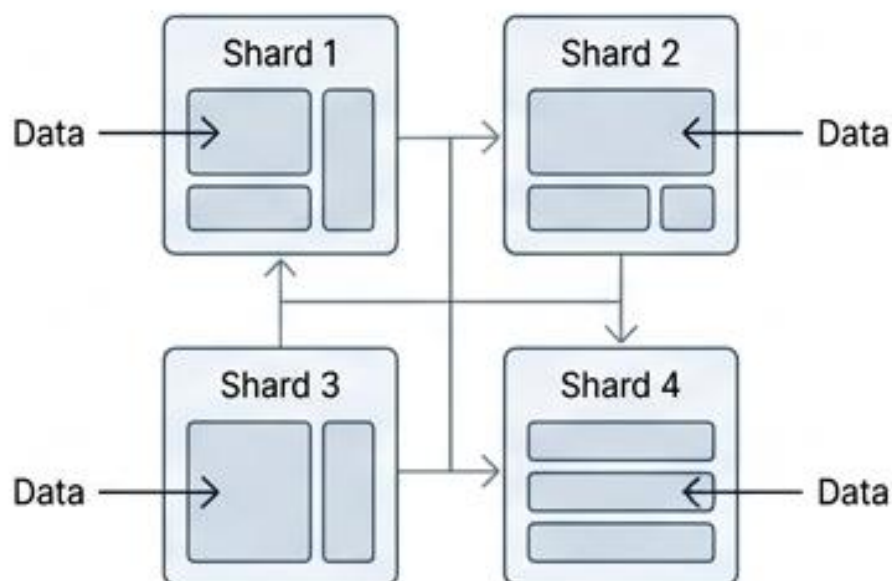
망분리, 개인정보 국외이전 제한 조직은 Managed 선택 불가 → 핵심 질문: '온프레미스 운영 인력 및 절차가 준비되어 있는가?'

선택 축 ② - 오픈소스의 3가지 층위 및 상용 비교

① 라이선스 종류 Apache 2.0(관대함) vs BSL 등(클라우드 서비스 제공 및 재판매 엄격히 제한) ② 오픈코어(Open-core) 모델 핵심 검색 엔진만 무료 제공, RBAC·백업·모니터링 등 필수 엔터프라이즈 기능은 유료화 ③ 커뮤니티 지속성 검증 소스 커밋 빈도, 이슈 대응 속도, 후원 기업의 재무 상태 (M&A가 매우 활발한 리스크 시장)		
구분	오픈소스 DBMS (Weaviate, Chroma, Qdrant 등)	상용 DBMS (Pinecone)
라이선스 비용	무료 (도입 장벽 낮음)	유료 (일부 제한적 Free Tier 제공)
커스터마이징	소스 공개를 통한 내부 환경 맞춤형 기능 개발 가능	제한적 제공 (전적으로 벤더 아키텍처에 의존)
보안 업데이트	커뮤니티 주도의 빠른 패치 (단, 미해결 이슈 방치 위험)	벤더 직접 지원 (보안 취약점 정보는 비공개 처리)
유지 및 관리	자체 관리 필수, 고도의 사내 인프라 기술력 요구	벤더의 직접 유지 관리 서비스 및 안정적 지원 보장

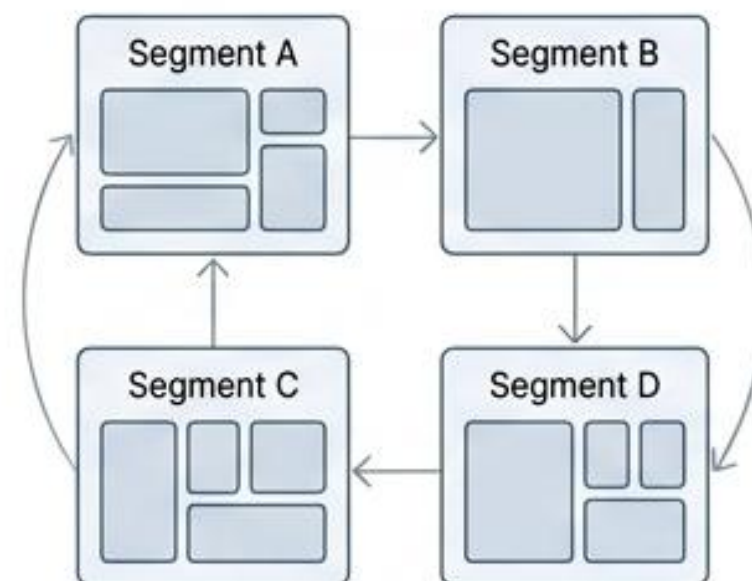
선택 축 ③ - 확장(Scale) 방식의 구조적 차이

Static 샤딩 (정적 할당)



- 작동 방식: 데이터를 미리 정의된 여러 샤드로 고정 분할
- 데이터 배치: 사전 계획된 고정 샤드에 데이터 수동/정적 분배
- 확장 한계성: 초기 설계 시 설정한 샤드 수에 의해 물리적 한계 봉착
- 운영 유연성: 샤드 구조 변경 및 확장 시 대규모 수동 데이터 재조정 필수

Dynamic Segment 배치 (동적 분할)



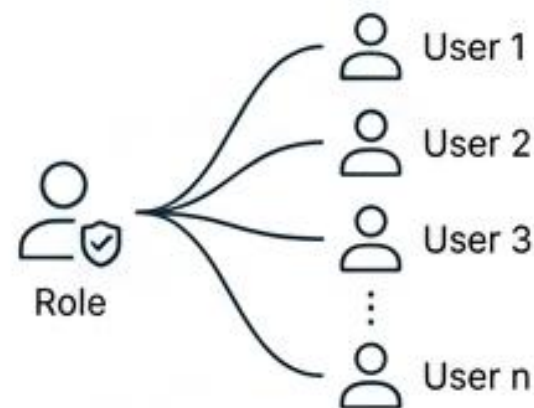
- 작동 방식: 시스템 부하 및 트래픽에 따라 필요 시 동적으로 세그먼트 생성
- 데이터 배치: 데이터 크기 및 시스템 리소스 부하에 따라 자동 분산 배치
- 확장 한계성: 트래픽 수요 및 데이터 증가에 맞춰 유연한 자동 수평 확장
- 운영 유연성: 시스템 차원의 지능적이고 자동화된 부하 분산(Load Balancing) 지원

[핵심 인사이트] 확장은 단순 성능 지표가 아닌 운영 부담의 문제. 데이터 재분배 시 대규모 재색인(Re-indexing) 및 서비스 중단 리스크 동반.

선택 축 ④ - 역할 기반 접근 제어 (RBAC)

RBAC의 정의 및 벤더별 지원 현황

- 개념 요약: 사용자 개별 설정 대신 '역할(Role)' 기준으로 권한을 일괄 통제하는 중앙집중형 보안 구조
- 완벽 지원: Pinecone, Milvus, Elasticsearch
- 미지원 / 제한적 지원: Weaviate, Chroma, Qdrant, pgvector



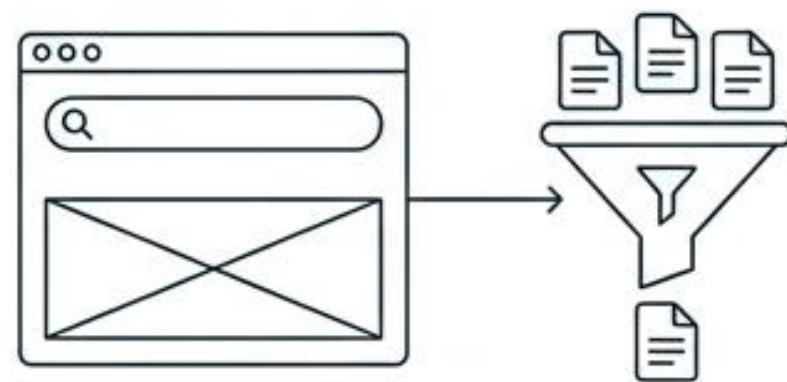
[DBMS 계층의 RBAC]

- 통제 목적: '누가 특정 컬렉션 전체에 읽기/쓰기를 수행할 수 있는가?'를 시스템적으로 통제



[RAG 애플리케이션 계층]

- 통제 목적: '특정 사용자가 인가받은 문서만 검색 결과에 포함되도록 필터링'
- 구현 원칙: 검색 시점에 메타데이터 필터(예: allowed_roles) 적용 필수
- 보안 주의사항: 클라이언트 측 필터 조건 전송 시 우회 해킹 위험
→ 반드시 서버 측에서 필터 조건 강제 주입 설계 필수



Vector DBMS 종류별 종합 특징 매트릭스

**FAISS 주의사항 - DBMS가 아닌 라이브러리. 영속성, 동시성 제어, 실시간 갱신 등 DB 필수 기능 부재로 운영계 시스템 부적합.*

분류	DB 명칭	오픈소스	로컬설치	지원 INDEX 구조	RBAC	핵심 특징 요약
Vector 전용	Weaviate	O	X	HNSW	-	자체 임베딩 기술 내장, SaaS 아키텍처 유리
Vector 전용	Chroma	O	X	HNSW	-	임베디드 모드 지원, 설치 없이 즉시 프로토타이핑
Vector 전용	Pinecone	X	O	폐쇄형 인덱스	O	RBAC 완벽 지원, 폐쇄망 설치 불가능한 엔터프라이 형
Vector 전용	Milvus	O	O	HNSW, FLAT, IVF	O	인덱스 다양화, 극강의 확장성, 운영 복잡도 최상
Vector 전용	Qdrant	O	X	HNSW	-	Managed 비용 최저수준, Rust 기반 성능 최적화
SQL 기반	pgvector	O	X	HNSW, IVFFlat	-	RDB JOIN 가능, ACID 트랜잭션 완벽 지원
NoSQL 기반	Elasticsearch	O	O	HNSW	O	Inverted Index 기반 텍스트 검색 기능 압도적 우위

후보 좁히기 5단계 프레임워크

1단계

배포 제약 조건 확인: 폐쇄망 환경 여부 및 데이터 국외 이전 제한 규제 검토

결과: 제한 시 *Managed* 전용
제품(*Pinecone* 등) 1차 소거

2단계

데이터 규모 확인: 시스템이 처리할 예상 전체 벡터 수 산정

결과: 수천만 건 이하의 소규모일 경우
RDB 통합형(*pgvector*) 유력 검토

3단계

검색 요건 확인: 단순 의미(Vector) 검색 외 정확한 키워드 검색 병행 필요성 판단

결과: 하이브리드 검색 내장 제품
우선순위 상향

4단계

보안 요건 확인: 조직 단위의 엄격한 RBAC 및 물리적 멀티테넌시(Multi-tenancy) 분리 필요성 점검



5단계

실측 및 PoC 검증: 최종 압축된 2~3개 후보군 대상 자체 비즈니스 데이터 기반 벤치마킹 진행



2026년 기준 실질적 제품 포지셔닝 지형

핵심 제품군	2026년 실질적 엔터프라이즈 생태계 포지션
pgvector	수천만 벡터 이하 환경의 기본 선택지. 벡터/관계형 단일 트랜잭션 처리로 운영 창구 일원화
Qdrant	성능·비용의 최적 균형점(Rust 기반). 복잡한 메타 필터링 강점, 신속한 PoC 도입 최적화
Milvus (Zilliz)	수억~수십억 메가 스케일 벡터 분산 처리 특화. 운영 복잡도로 인해 전담 인프라 인력 필수
Weaviate	벡터+키워드+메타 융합 하이브리드 검색 특화. 멀티테넌시 구조 우수, B2B SaaS에 적합
Pinecone	완전 관리형(Managed)의 대표 주자. 운영 부담 최소화, 단 대규모 확장 시 비용 급증, 폐쇄망 불가
Chroma	프로토타이핑 및 AI 모델 학습용 샌드박스 최적화. 엔터프라이즈급 대규모 쓰기 부하에는 부적합
Elasticsearch	기존 로그/검색 인프라 보유 조직에 벡터 기능을 추가 통합하는 가장 안전하고 합리적 선택

차세대 주요 선택 축 ⑤~⑦

[선택 축 ⑤] 하이브리드 검색 내장 여부

- 핵심 고려사항: DB 내부 자체 융합 vs 앱 단 직접 융합(RRF) 방식에 따른 개발 공수 차이 극심
- 체크리스트: 희소 벡터(SPLADE) 지원 및 다중 벡터(CoBERT) 아키텍처 호환 여부 점검 필수



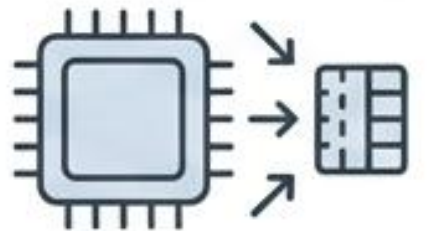
[선택 축 ⑥] 멀티테넌시 (Multi-tenancy) 격리 구조

- 핵심 고려사항: B2B 서비스 및 부서별 데이터 격리 시 성능 저하 없는 논리/물리적 분리 지원 여부
- 한계 경고: 테넌트가 수천 개 도달 시, 단순 '컬렉션 무한 생성' 방식의 아키텍처는 물리적 한계 직면



[선택 축 ⑦] 메모리 비용 및 양자화 (Quantization)

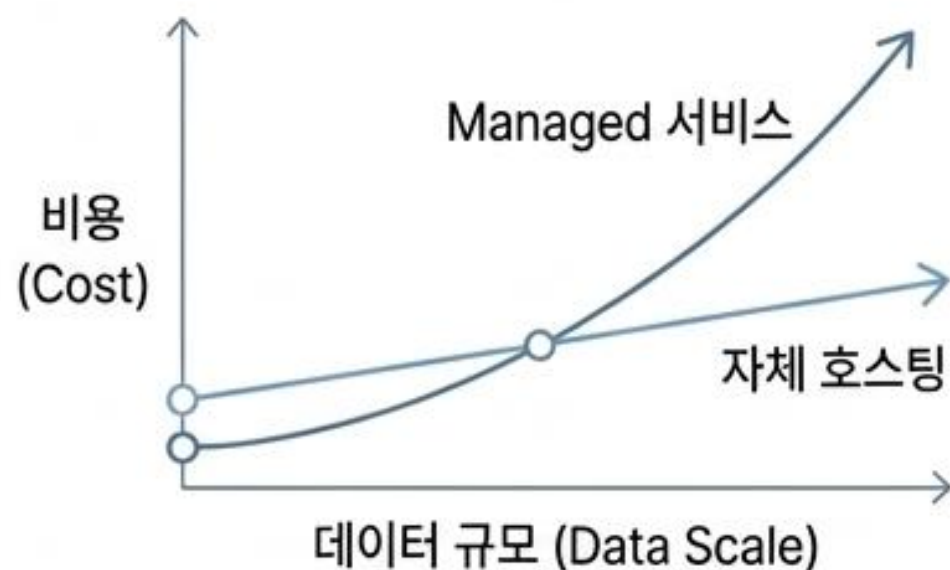
- 핵심 고려사항: 스칼라(Scalar) 및 이진(Binary) 양자화 기술, 디스크 기반 인덱스 지원 여부
- 도입 영향: 대규모 도입 시 총비용을 좌우하는 핵심. 동일 예산 대비 더 큰 데이터 처리량 결정



규모에 따른 비용 구조 역전 현상

비용 산정 시 흔히 범하는 함정

- 프로토타입 단계의 단순 Managed 서비스 월 구독료만 보고 도입을 결정하는 오류 빈번
- 데이터 규모 1억 건 이상 확대 시, Managed 솔루션과 자체 호스팅 간 TCO(총소유비용) 수 배 격차 발생



데이터 규모별 권장 솔루션 및 유의점

데이터 규모	일반적 비용 효율 판단	설계 유의점
~ 100만 벡터	Chroma, pgvector, 또는 각 제품의 무료 티어 활용	전용 Vector DB의 성급한 독립 구축은 대부분 과잉 설계(Over-engineering)
~ 1천만 벡터	pgvector 또는 관리형(Managed) 서비스 모두 합리적	기존 내부 인력의 PostgreSQL 역량 보유 시 pgvector 도입 압승
1억 벡터 이상	자체 호스팅(Milvus, Qdrant 등)이 장기 TCO 측면 유리	전담 인프라 운영 및 유지보수 인력 비용 별도 산정 필수

[핵심 원칙] 총 소유 비용(TCO) 산정 시 '인프라 사용료 + 사내 운영 인력 비용' 반드시 합산

벤치마크의 한계와 성공적인 PoC 설계 가이드



[체크 1] 공개 벤치마크의 맹점 인지

- 공개 데이터는 특정 벤더 아키텍처에 유리한 조건으로 세팅됨
- 데이터 분포, 차원 수, 필터 조건, 동시 요청 수에 따라 실제 순위 상이함



[체크 2] 정교한 자체 평가셋 구축 (PoC 성공의 8할)

- 비즈니스 실무 기반 질문 50~100건 추출 및 정답 문서 라벨링 사전 완료 필수



[체크 3] 통제된 동일 조건 적용

- 임베딩 모델, 차원 크기, 텍스트 청킹(Chunking) 방식을 모든 후보군에 고정 적용



[체크 4] 복합 지표 동시 측정

- 단순 처리 속도(QPS, p95)와 검색 품질(Recall@k, nDCG) 교차 검증
- 실무 환경과 동일한 권한, 기간, 부서 메타 필터 조건 주입 후 성능 측정



[체크 5] 극단적 운영 시나리오 검증

- 대량 일괄 색인, 전체 인덱스 재구축, 장애 복구 타임, 무중단 업그레이드 테스트 수행

전환 비용(Lock-in) 산정 및 데이터 아키텍처 전략

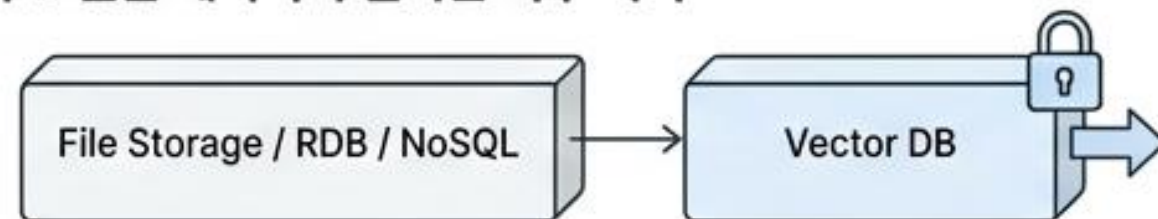
DB 교체 시 발생하는 숨은 전환 비용 리스크



- ✓ **물리적 시간:** 수억 벡터 규모 이관 및 전면 재구축 시 수 시간~수 일 소요
- ✓ **코드 종속성:** 질의 API 체계 및 필터 문법 차이로 인한 대규모 애플리케이션 수정 불가피
- ✓ **품질 재현:** 제품별 하이브리드 스코어링 로직 차이로 검색 품질 재튜닝 공수 폭증

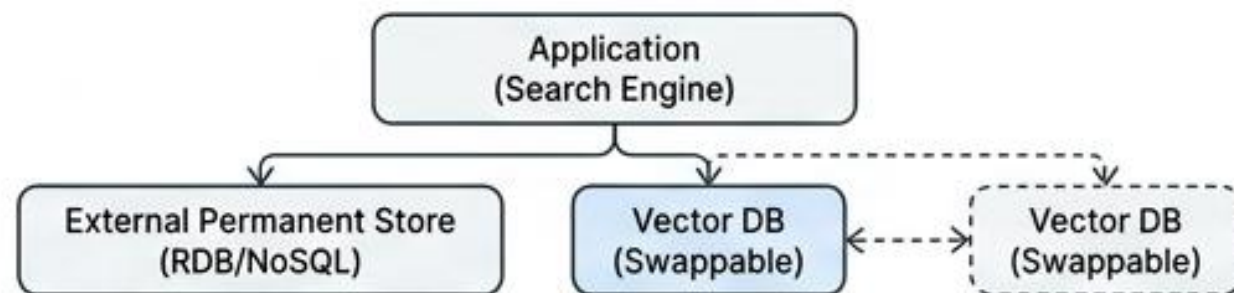
Lock-in 완화를 위한 2단계 아키텍처 방어 전략

• 원칙 1. 원본 데이터의 철저한 외부 격리



- 원문 데이터와 텍스트 청크를 절대 Vector DB 내부에 단독 유통하지 않음
- 파일 스토리지, RDB, NoSQL 등 기존 영구 저장소에 마스터 데이터 반드시 이중 보관

• 원칙 2. 조합형 아키텍처 채택



- 단일 솔루션 의존 배제, '여러 DBMS의 조합' 패턴 채택
- 검색 엔진과 영구 저장소의 역할을 명확히 분리하여 유연한 교체 환경 사전 구축

특정 벤더 종속을 탈피하고 유연한 이관 경로를 확보하는 것이 최종 아키텍처 설계의 목표